NYC Taxi Mobility Dashboard: Technical Report

Summative assignment · Team: **Junior Nyamu and Ahmed Osman** · Data: NYC TLC yellow taxi trips, January 2019

1. Problem framing and dataset analysis

Context. We investigate when and where urban movement occurs in NYC in the context of the three official TLC files: the fact table (7,696,617 rows of yellow-taxes trips, with timestamps, distance, rate code, itemised fare, pickup zone ID, dropoff zone ID); the zone id to zone, borough, and service zone file (which maps 265 zone IDs to zones, boroughs and service zones; zones 264 and 265 are missing); and the zone shape file (263 zones, with a zone shape) (zone 264 and 265 are not present). The objective of the solution is to create an interactive platform which allows a planner to filter and compare trips in terms of time, distance, fare and location. On average, a trip is 2.82 miles and $12.14 for making the trip. 72% of trips are paid by card after cleaning.

Data challenges. The raw data contains physically impossible and suspicious records. Our cleaning step applies 17 ordered rules; the first rule a row breaks is its recorded reason, so every excluded row is counted once. It removed 249,573 rows (3.24%), leaving 7,447,044. The rules for exact duplicates, missing required fields and unknown zone IDs matched nothing in this month.

Rule (reason excluded)	Rows	% raw
Passenger count not 1-6	143,693	1.87
Distance zero or negative	48,834	0.63
Duration over 6 hours	20,463	0.27
Fare per mile over $100	10,839	0.14
Fare zero or negative	6,720	0.09
Dropoff not after pickup	6,554	0.09
Distance under 0.1 miles	5,870	0.08

Rule (reason excluded)	Rows	% raw
Speed over 80 mph	5,462	0.07
Pickup outside January 2019	537	<0.01
Tip over 3x fare (card)	475	<0.01
Invalid rate code	91	<0.01
Distance over 100 miles	31	<0.01
Fare over $500	4	<0.01

Table 1. Rows removed by each rule (percent of the raw rows). Full log: logs/excluded_records.csv and logs/exclusion_summary.csv.

Assumptions. Each threshold represents a type of possible taxi trip: Duration up to 6 hours; Distance 0.1 to 100 miles; Fare up to $500; Average speed up to 80 mph; Fare per mile up to $100 (raw, up to $41,500); For card payments, tip must be at least three times the fare (raw, at least 450,000%). Numbers in the range of 1 to 6 are considered valid, numbers outside of this range are invalid. This is our largest exclusion (143,693, .87% of raw) and a judgement call: a count of zero may represent an unfilled driver entry instead of a poor trip, which means that trip counts could be slightly understated. If no cash tips are used, the tip percentage is not calculated for these payments.

Derived features. duration_min and avg_speed_mph (distance divided by hours) expose congestion; fare_per_mile measures pricing efficiency and doubles as an outlier detector; tip_pct captures customer behaviour; pickup_hour, day_of_week and is_weekend make temporal patterns cheap to filter and group.

Unexpected observation. Pickups are extremely concentrated: 6,769,137 of 7,447,044 (90.9%) are in Manhattan, while Staten Island has 279 and Newark airport (EWR) 23. Averages over a handful of trips are unreliable, so the map applies a minimum-trips threshold per zone and greys out zones below it instead of colouring them from tiny samples. A related quirk: 127,467 trips (1.7%) both start and end in zone 264, which the TLC lookup lists with an unknown borough. Their fares and distances are valid, so we keep them, but they cannot be placed on the map.

2. System architecture and design decisions

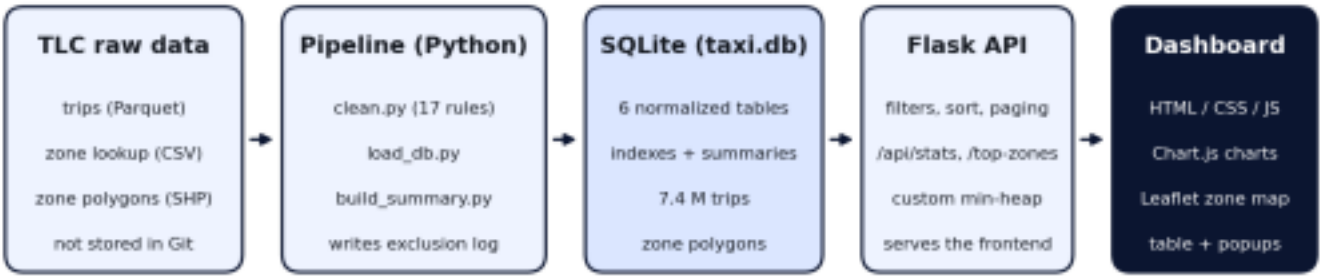


Figure 1. Architecture: raw files flow through the cleaning pipeline into SQLite, are served by a Flask API and shown in the browser dashboard.

Stack. Python with pandas and PyArrow handles 7.7 million rows in a columnar format; SQLite gives a zero-configuration, single-file database that ships easily as a dump and suits read-heavy analytics; Flask provides a small JSON API that also serves the static dashboard, so one command runs everything; plain JavaScript with Chart.js and Leaflet needs no build step.

Schema. Six normalized tables: borough to zone to zone_boundary (one polygon per zone), lookup tables payment_type and rate_code, and the fact table trip with foreign keys to zone (pickup and dropoff), payment type and rate code. Indexes on pickup time, pickup zone, dropoff zone, pickup hour, payment type and zone borough back each dashboard filter and GROUP BY. API sort and metric names are whitelisted and all values are bound as SQL parameters.

Trade-offs.

- **Derived features stored in trip.** Slightly denormalized, which costs space and could drift from the raw columns, but queries stay fast; the table is always rebuilt from the pipeline.
- **Summary tables vs live queries.** Unfiltered charts read tiny precomputed tables; filtered charts scan all 7.4 million trips (a few seconds) instead of precomputing every filter combination.
- **SQLite vs PostgreSQL.** We chose simplicity and portability over concurrency and PostGIS; polygons are stored as GeoJSON text.
- **Speed metric.** A plain average of per-trip speeds is dominated by very short trips and rounding noise, so speed is total miles divided by total hours. Trip-table paging uses LIMIT/OFFSET, which is simple but slows on very deep pages.

3. Algorithmic logic and data structures

Problem. Rank the k busiest pickup zones for any filter combination (the Top zones chart and /api/top-zones).

Approach. We implemented a binary min-heap by hand (an array where item i has parent (i-1)//2 and children 2i+1 and 2i+2) and keep the best k zones seen so far in it. The root is the weakest of them, so a new zone only has to beat the root to get in. No heap, Counter or sort function is used.

```
TOP-K(zone_counts, k):
 heap <- empty min-heap ordered by count
 for each (zone, count) in zone_counts:
 if size(heap) < k: push(heap, (count, zone)) # O(log k)
 else if count > root(heap): pop(heap); push(heap, (count, zone))
 result <- []
 while heap not empty: append(result, pop(heap)) # ascending
 return reverse(result) # biggest first

push: append at the end, then SIFT-UP while smaller than its parent
pop: take the root, move the last item to the root, then SIFT-DOWN
 (swap with the smaller child until neither child is smaller)
```

Complexity. Time **O(n log k)**: each of the n zones costs at most one pop and one push, and each is O(log k) because the heap has height log k. Emptying the heap costs O(k log k) and the reversal O(k), both within O(n log k) since k is at most n. Space **O(k)**: the heap never holds more than k items. Sorting all zones would cost O(n log n), so the heap wins when k is much smaller than n. Here n is at most 265 zones and the API limits k to 1 to 100, so either is fast; the heap is the right choice if the same ranking is later applied to a larger key space such as origin-destination pairs.

4. Insights and interpretation

Insight 1. Demand and speed move in opposite directions. Derived from /api/stats/by-hour: GROUP BY pickup_hour with COUNT(*) and SUM(distance) / (SUM(duration_min) / 60), drawn as bars (trips) and a line (mph) on the dashboard. Trips peak at 18:00 (499,957) and bottom out at 04:00 (58,441), 8.6 times fewer. Speed is slowest at 09:00 (10.9 mph) and fastest + at 05:00 (22.9 mph), about twice as fast. Across the 24 hours the correlation between trips and speed is r = -0.85. Meaning: taxis crawl through the morning and evening rush, so the same trip takes about half as long at 05:00 as at 09:00. The 05:00 peak is probably lifted by long airport and highway trips, and correlation alone does not prove demand causes the slowdown. For planners, the slowest hour (09:00) is not the busiest (18:00), so congestion measures must target the morning.

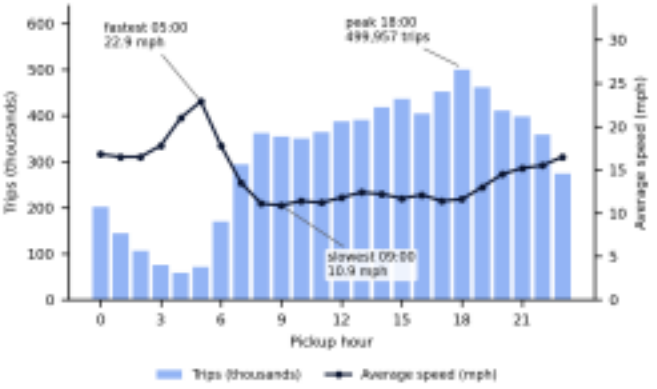
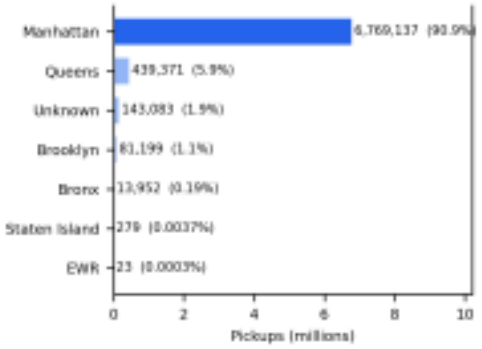


Figure 2. Trips and average speed by pickup hour (all 7.4 million cleaned trips).



Insight 2. Yellow-cab service is concentrated in Manhattan. Derived from a join of trip, zone and borough grouped by pickup borough (pipeline/check_db.py, /api/insights/borough) and the pickups map. Manhattan has 90.9% of pickups, Queens 5.9%, zones with an unknown borough 1.9%, Brooklyn 1.1%, the Bronx 0.19% and Staten Island 0.004%. Trips also differ in kind: the average Queens pickup is 11.69 miles and $35.63, against 2.22 miles and $10.54 in Manhattan, consistent with the two airports in Queens. Meaning: yellow cabs are essentially a Manhattan core service, and outer-borough trips are rarer and much longer. Other services such as green taxis and app-based rides fill the gap, so planning from this data alone would overstate Manhattan's share of total demand, and any The per-zone average outside Manhattan rests on a few trips.

Figure 3. Pickups by borough.

Insight 3. Weekdays and weekends carry similar volumes but different rhythms. This is calculated from /api/insights/hour-weekend: GROUP BY is_weekend (23 weekdays, 8 weekend days) to compare the trips per day. Average weekday trips are 244,986 and average weekend trips are 226,547 (7.5% less), but the shapes are different. At 08:00 a weekday has 13,771 trips against 5,651 (2.4 times) and traffic moves at 10.4 mph against 18.7 mph. The weekend curve is flat from 12:00 to 18:00 (13,022 to 13,806 trips per day), where weekday volume climbs to 16,935 at 18:00. That changes after midnight: 11,168 trips per weekend day compared to 4,875 per weekday at 00:00. The meaning is that the demand on weekdays is commute-induced and subject to major peaks during rush hours and lulls elsewhere, while the demand on weekends is leisure-based, featuring an even afternoon, strong late night demand and faster speeds. Taxi supply should then match the demand on weekdays and on weekends, when people are going out for a night. Cautions: A journey is considered weekend by the day of pickup, therefore Friday and Saturday nights count as two weekdays and two weekend days; and two public holidays (1st and 21st January) are weekdays and weaken the weekday figures.

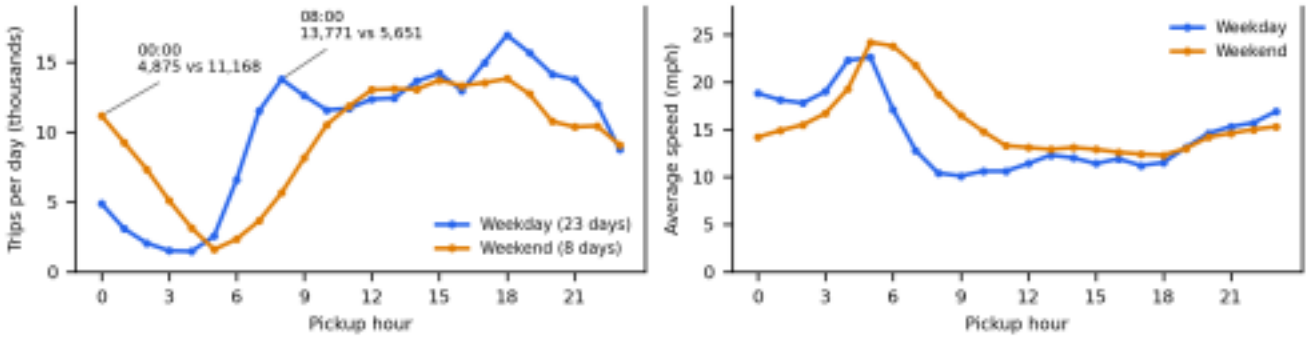


Figure 4. Trips per day (left) and average speed (right) by pickup hour, weekdays versus weekend days.

5. Reflection and future work

Technical challenges. Scale: 7.4 million rows required chunked loading (500,000 rows at a time) and summary tables to keep the dashboard responsive. Realism: a plain average of per-trip speeds gave implausible hourly values, which we replaced with a time-weighted speed. Tooling: a UTF-16 requirements file and a .gitignore corrupted by a UTF-16 append caused avoidable failures; GitHub rejected a 128 MB data file (limit 100 MB), so raw data stays out of Git and the README documents the download; and the zipped shapefile could not be opened on Windows, so the README extracts it first.

Team challenges. As a team of two, we spent our first few calls ideating, agreeing on a structure and working out what the assignment expected before building, which settled the design early but left less time for implementation and testing.

Next steps for a real product.

Move to PostgreSQL with PostGIS for concurrent users and spatial queries; replace OFFSET paging with keyset paging and cache or materialize filtered aggregates; add more months and the green-taxi and for-hire data to correct the Manhattan bias; add origin-destination flow maps and clustering of zones by hourly profile; and deploy with automated tests and continuous integration.

AI assistance: Claude was used for debugging, code review and drafting this report, as recorded in our AI usage log.